

Progetto di Ingegneria del Software

Gestione Iniziative e Bachecca Eventi

Ruggeri Davide 747653 - Musatti Simone 746400

Anno Accademico 2025-2026

Indice

1	Introduzione	3
1.1	Scopo del Progetto	3
2	Versione 1	4
2.1	Obiettivi e Requisiti	4
2.2	Casi d'Uso	4
2.2.1	Elenco Casi d'Uso	4
2.3	Architettura e Design	4
2.3.1	Classi Principali (V1)	5
2.3.2	Diagramma delle Classi	5
3	Versione 2	8
3.1	Obiettivi e Requisiti	8
3.2	Casi d'Uso (Aggiunte rispetto a V1)	8
3.2.1	Regole di Validazione delle Proposte	8
3.2.2	Diagramma UML dei Casi d'Uso (V2)	9
3.3	Architettura e Design (Aggiunte rispetto a V1)	9
3.3.1	Nuove Classi (V2)	9
3.3.2	Diagramma delle Classi (Delta V2)	9
4	Versione 3	12
4.1	Obiettivi e Requisiti	12
4.2	Nuove Funzionalità (V3)	12
4.3	Casi d'Uso (Aggiunte rispetto a V2)	12
4.4	Architettura e Design (Aggiunte rispetto a V2)	13
4.4.1	Pattern Observer	13
4.4.2	Nuove Classi (V3)	13
4.4.3	Metodi Principali Aggiunti	13
4.4.4	Diagramma delle Classi (Delta V3 - Pattern Observer)	13
5	Versione 4	16
5.1	Obiettivi e Requisiti	16
5.2	Nuove Funzionalità (V4)	16
5.3	Casi d'Uso (Aggiornamento V4)	16
5.4	Architettura e Design (Aggiunte rispetto a V3)	16
5.4.1	Nuovo Stato dell'Enum	16
5.4.2	Metodi Principali Aggiunti	17
5.4.3	Diagramma delle Classi	17

6	Versione 5	20
6.1	Obiettivi e Requisiti	20
6.2	Modalità Batch	20
6.2.1	Formato del File	20
6.2.2	Esecuzione	20
6.2.3	Output Batch	21
6.3	Architettura (V5)	21
6.4	Diagramma Globale dei Casi d'Uso	21
6.5	Diagramma Globale delle Classi	21
7	Manuale Utente	24
7.1	Installazione e Avvio	24
7.1.1	Prerequisiti di Sistema	24
7.1.2	Struttura di Salvataggio Dati	24
7.1.3	Avvio tramite Ambiente di Sviluppo (IDE)	24
7.1.4	Avvio tramite Riga di Comando (Terminale)	25
7.1.5	Avvio speciale con Importazione Batch (V5)	25
7.2	Operatività e Manuale d'Uso	25
7.2.1	Primo Accesso al Sistema (Login e Registrazione)	25
7.2.2	Menu del Configuratore (Amministratore)	26
7.2.3	Menu del Fruitore (Utente Base)	27

Capitolo 1

Introduzione

1.1 Scopo del Progetto

Il presente documento raggruppa la documentazione di progetto relativa all'applicazione software per la **gestione di iniziative e bacheca eventi**, sviluppata secondo un approccio incrementale in cinque versioni (v1-v5).

Come richiesto dalle specifiche, il documento illustra l'evoluzione del sistema, riportando per ogni versione:

- I **Casi d'Uso** (testuali e diagrammi UML).
- I **Diagrammi delle Classi** (UML), evidenziando via via le componenti modificate o aggiunte.
- Le scelte progettuali adottate (es. design dei controller, serializzazione personalizzata, ecc.).

L'ultimo capitolo riporta invece i diagrammi UML globali (senza evidenziazione dei delta) e il manuale d'uso definitivo.

Capitolo 2

Versione 1

2.1 Obiettivi e Requisiti

La prima versione introduce il dominio del sistema: **Categorie**, **Campi** (Base, Comuni, Specifici) e l'attore principale, il **Configuratore**. L'obiettivo è costruire una struttura flessibile per rappresentare qualunque tipo di iniziativa tramite una gerarchia di categorie configurabile.

2.2 Casi d'Uso

2.2.1 Elenco Casi d'Uso

1. **Login / Cambio Password:** Il configuratore inserisce le credenziali predefinite (*admin/admin*) al primo accesso, costretto poi a impostare una nuova password personale.
2. **Gestire Campi Base:** Il configuratore definisce i campi comuni fissi di sistema all'avvio.
3. **Gestire Campi Comuni:** Il configuratore aggiunge, modifica (obbligatorietà) o rimuove campi condivisi da tutte le categorie.
4. **Creare Categoria:** Il configuratore crea una nuova categoria con nome, descrizione ed eventuali campi specifici; può essere figlia di una categoria esistente.
5. **Modificare Categoria:** Il configuratore aggiunge o rimuove campi specifici a una categoria esistente.
6. **Rimuovere Categoria:** Il configuratore elimina una categoria e i relativi campi.
7. **Visualizzare Categorie:** Il configuratore naviga l'albero delle categorie e relativi campi.

2.3 Architettura e Design

Il pattern architetturale adottato separa nettamente i dati (package **model**) dalla logica di business (package **controller**) e dalla presentazione (package **view**).

2.3.1 Classi Principali (V1)

- **Utente** (abstract): base per tutti gli attori del sistema.
- **Configuratore** extends **Utente**: unico attore in V1.
- **Categoria**: struttura ad albero (lista di sottocategorie), contiene una mappa di **Campo**.
- **Campo**: nome, descrizione, tipo (**TipoCampo**: **STRINGA**, **INTERO**, **BOOLEANO**, **DATA**, **ORA**), flag obbligatorio.
- **GestoreCategorie**: logica di creazione, modifica e navigazione dell'albero.
- **GestoreSessione**: gestione login, logout e stato sessione corrente.
- **GestoreFile**: persistenza su file JSON (via libreria Gson).
- **MainCLI**: interfaccia a riga di comando; delega tutta la logica ai controller.

2.3.2 Diagramma delle Classi

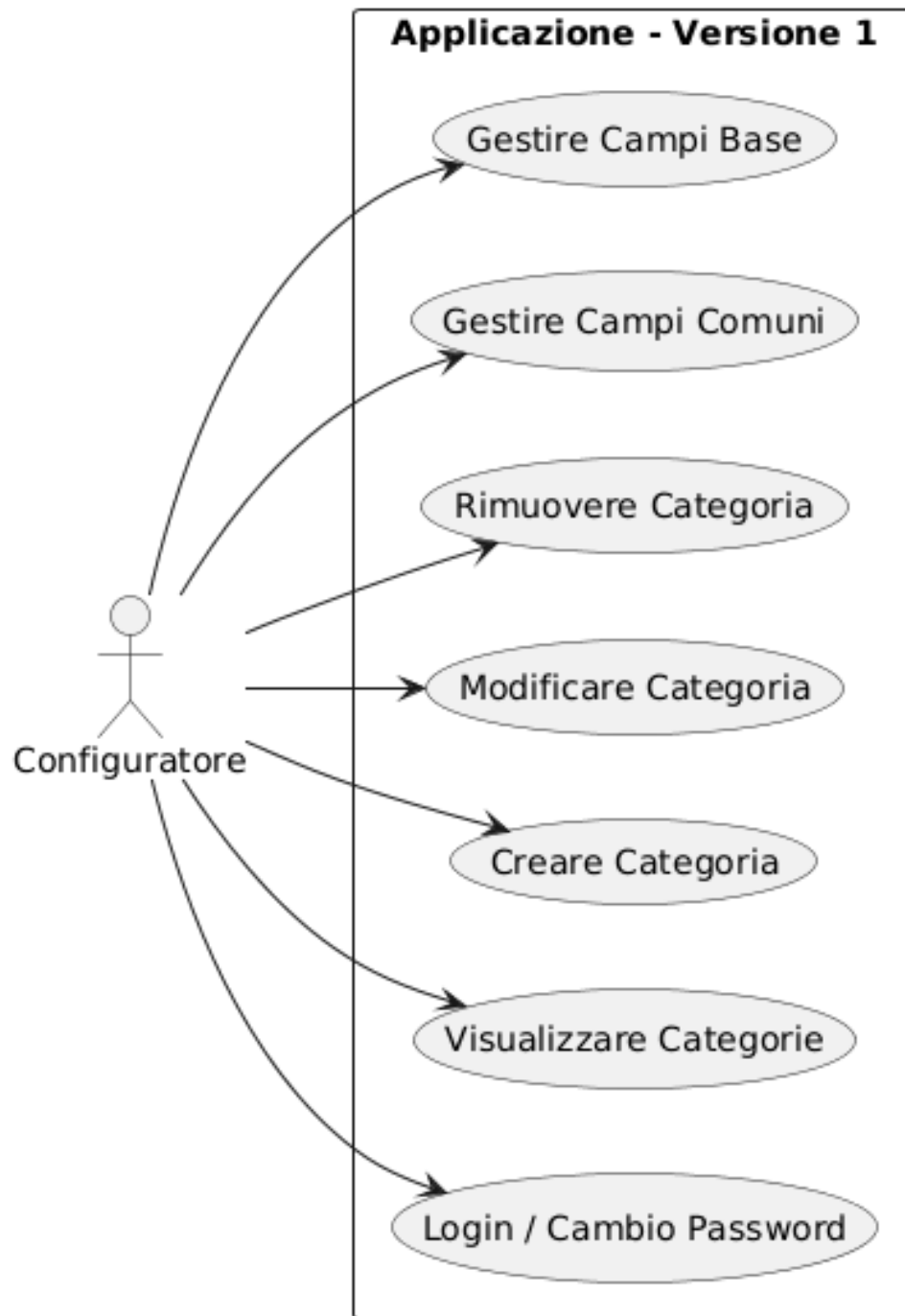


Figura 2.1: Diagramma dei Casi d'Uso - Versione 1

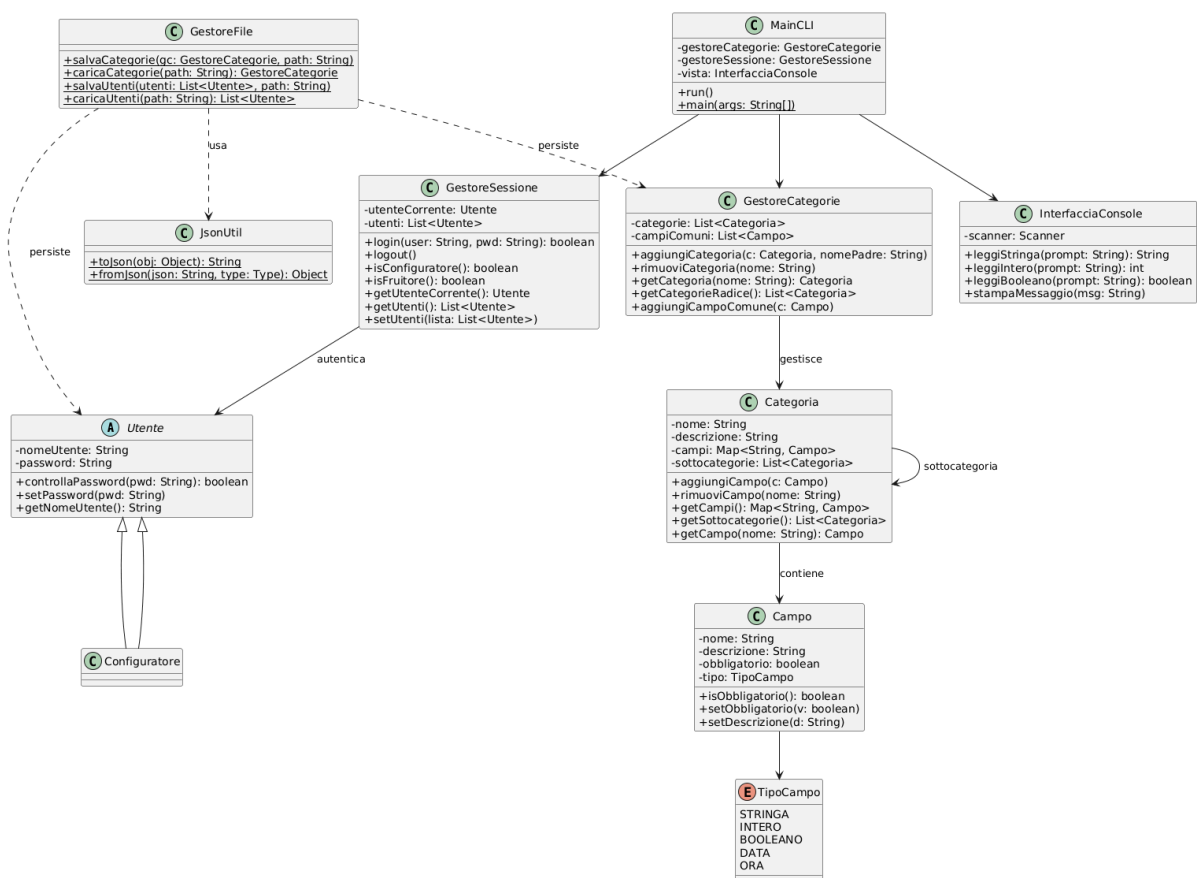


Figura 2.2: Diagramma delle Classi - Versione 1

Capitolo 3

Versione 2

3.1 Obiettivi e Requisiti

La seconda versione introduce la **Proposta di Iniziativa**: il Configuratore può ora proporre eventi basati sulle categorie definite in V1, validarli e pubblicarli sulla **Bacheca** di sistema.

3.2 Casi d'Uso (Aggiunte rispetto a V1)

I casi d'uso base del Configuratore (login, gestione categorie e campi) restano invariati. Si aggiungono:

8. **Creare Proposta**: Il configuratore seleziona una categoria, compila il modulo con tutti i campi obbligatori (Titolo, Data, Termine ultimo iscrizione, Numero partecipanti, Luogo, Ora, Quota). Il sistema verifica le date e assegna lo stato **VALIDA**.
9. **Pubblicare Proposta in Bacheca**: Il configuratore rende pubblica una proposta valida, che passa allo stato **APERTA**.
10. **Visualizzare Bacheca**: Il configuratore visualizza tutte le proposte aperte, raggruppate per categoria.

3.2.1 Regole di Validazione delle Proposte

- Il **Termine ultimo di iscrizione** deve essere una data futura.
- La **Data** dell'evento deve essere almeno due giorni dopo il termine di iscrizione.
- Tutti i campi obbligatori (marcati con *) devono essere compilati.

3.2.2 Diagramma UML dei Casi d'Uso (V2)

3.3 Architettura e Design (Aggiunte rispetto a V1)

3.3.1 Nuove Classi (V2)

- **Proposta**: contiene la categoria di riferimento, una mappa di valori per i campi, lo stato (**StatoProposta**) e una lista di iscritti (**username**).
- **StatoProposta** (enum): **VALIDA**, **APERTA**, **CONFERMATA**, **ANNULLATA**, **CONCLUSA**.
- **Bacheca**: mappa le proposte per nome categoria. Fornisce metodi di filtro per stato.
- **GestoreProposte**: coordina validazione, pubblicazione e controllo scadenze.

3.3.2 Diagramma delle Classi (Delta V2)

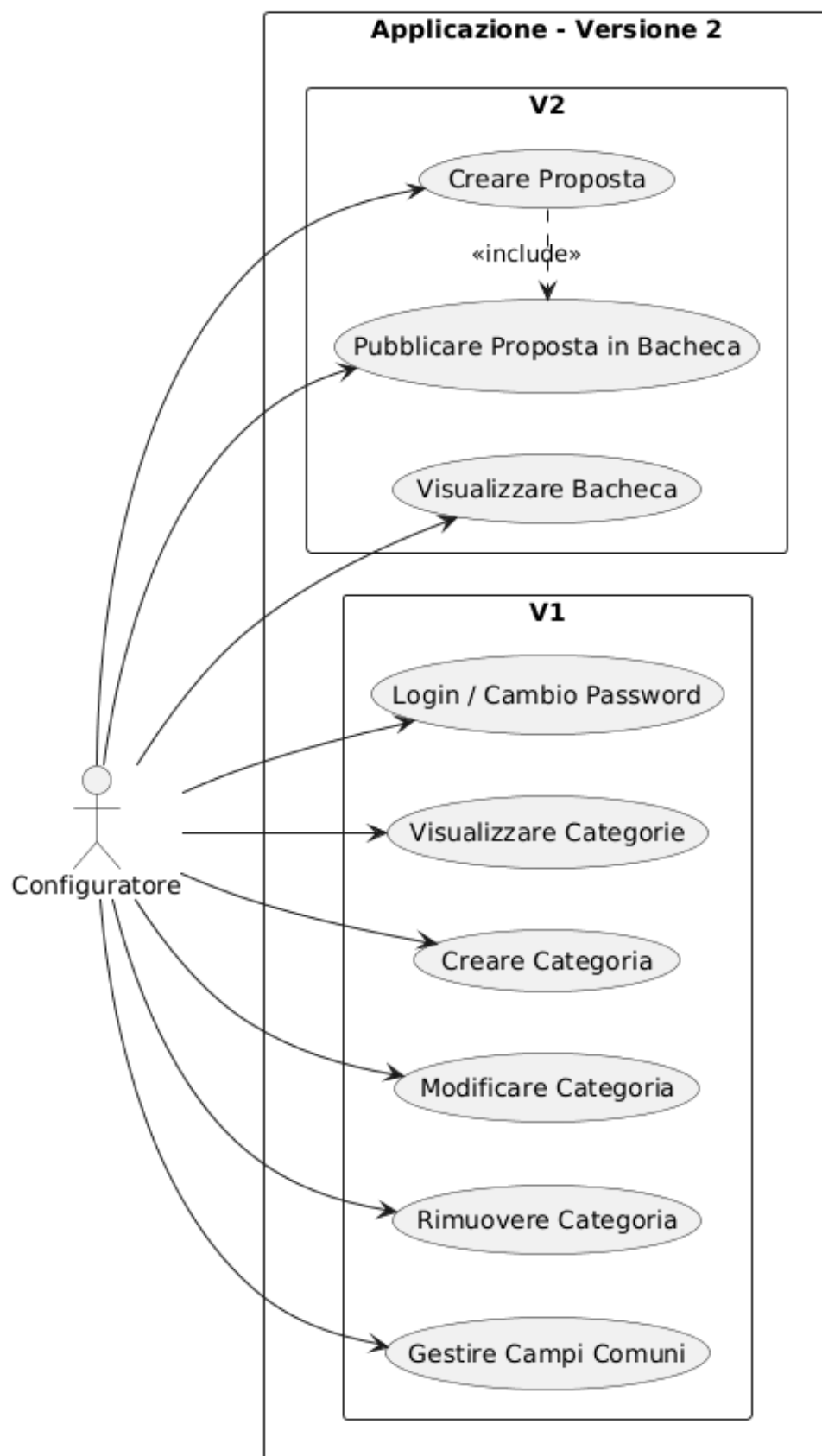


Figura 3.1: Diagramma dei Casi d'Uso - Versione 2

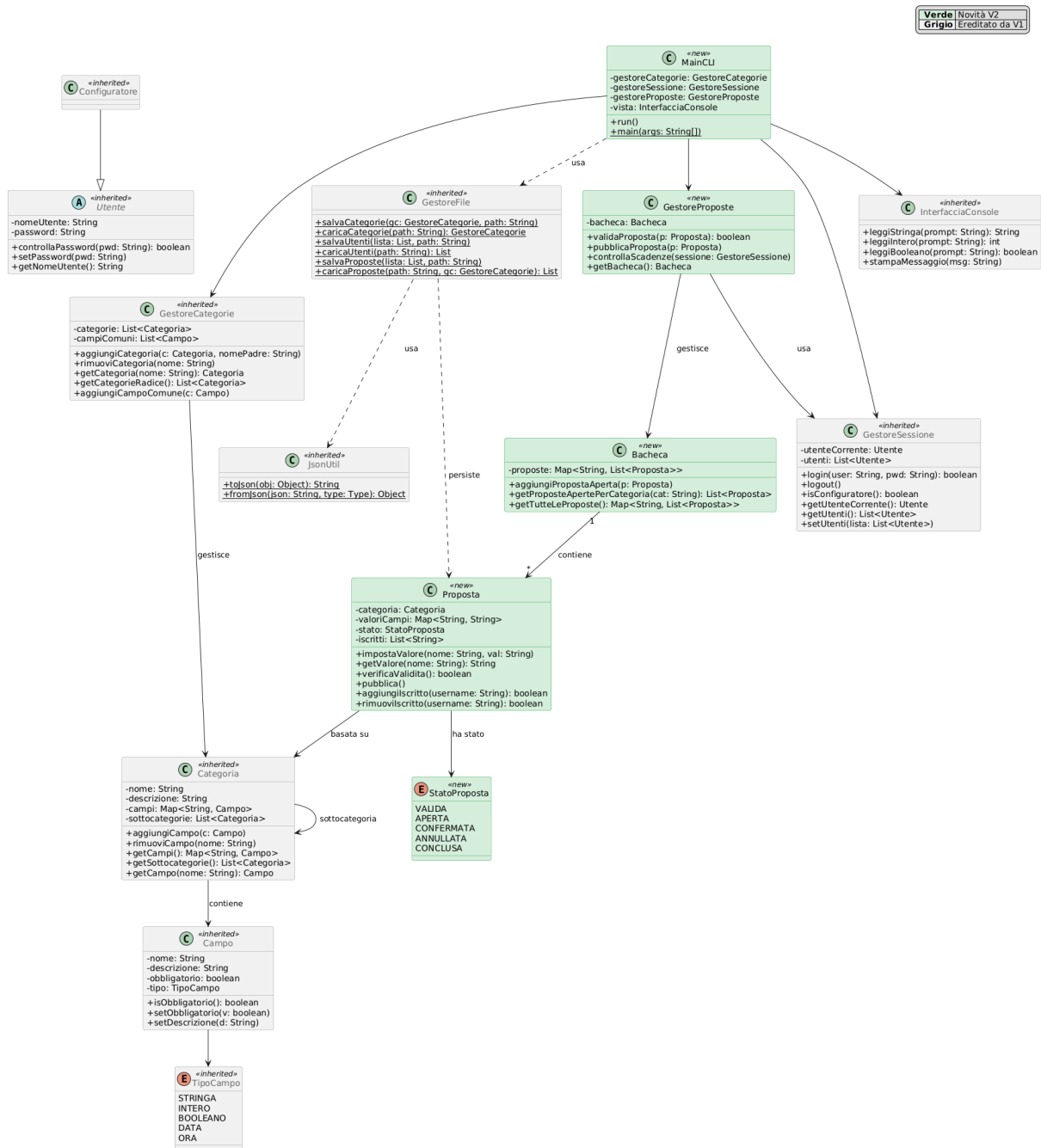


Figura 3.2: Diagramma delle Classi (Delta evidenziati) - Versione 2

Capitolo 4

Versione 3

4.1 Obiettivi e Requisiti

La terza versione introduce il ruolo del **Fruitore**, che può registrarsi al sistema, visualizzare le proposte aperte in Bacheca, iscriversi agli eventi di suo interesse e ricevere aggiornamenti automatici tramite il **pattern Observer** quando lo stato di un evento cambia.

4.2 Nuove Funzionalità (V3)

1. **Registrazione Fruitore:** Al primo accesso con uno username non riconosciuto, il sistema offre la possibilità di creare un account Fruitore.
2. **Iscrizione a Proposta:** Il Fruitore può iscriversi a qualsiasi proposta in stato APERTA, entro il limite massimo di partecipanti.
3. **Ritiro Iscrizione:** Il Fruitore può disdire la propria iscrizione finché la proposta è ancora APERTA e non è scaduto il termine utile.
4. **Area Personale (Notifiche):** Il Fruitore visualizza e cancella i messaggi ricevuti automaticamente dal sistema quando una proposta a cui è iscritto cambia stato.
5. **Gestione Automatica degli Stati:** Il sistema verifica le scadenze al login e aggiorna automaticamente le proposte: APERTA → CONFERMATA (target raggiunto) o ANNULLATA (target non raggiunto); CONFERMATA → CONCLUSA (data evento passata).

4.3 Casi d'Uso (Aggiunte rispetto a V2)

- **Registrazione e Login Fruitore**
- **Visualizzazione Bacheca** (proposte APERTE)
- **Iscrizione / Ritiro da Proposta Aperta**
- **Gestione Notifiche personali**

4.4 Architettura e Design (Aggiunte rispetto a V2)

4.4.1 Pattern Observer

Il pattern Observer decoupla il meccanismo di notifica dalla logica di gestione degli stati:

- **Observer** (interface): definisce `update(String messaggio)`.
- **Observable** (interface): definisce `addObserver`, `removeObserver`, `notifyObservers`.
- **Fruitore implements Observer**: memorizza i messaggi ricevuti in una lista `notifiche`.
- **Proposta implements Observable**: notifica tutti gli observer registrati ogniqualvolta `setStato()` aggiorna lo stato.

Nota tecnica: La lista `observers` di `Proposta` è marcata `transient` per evitare problemi di serializzazione JSON. Gli observer vengono ricollegati a runtime da `GestoreProposte.collega()` prima di ogni cambio di stato.

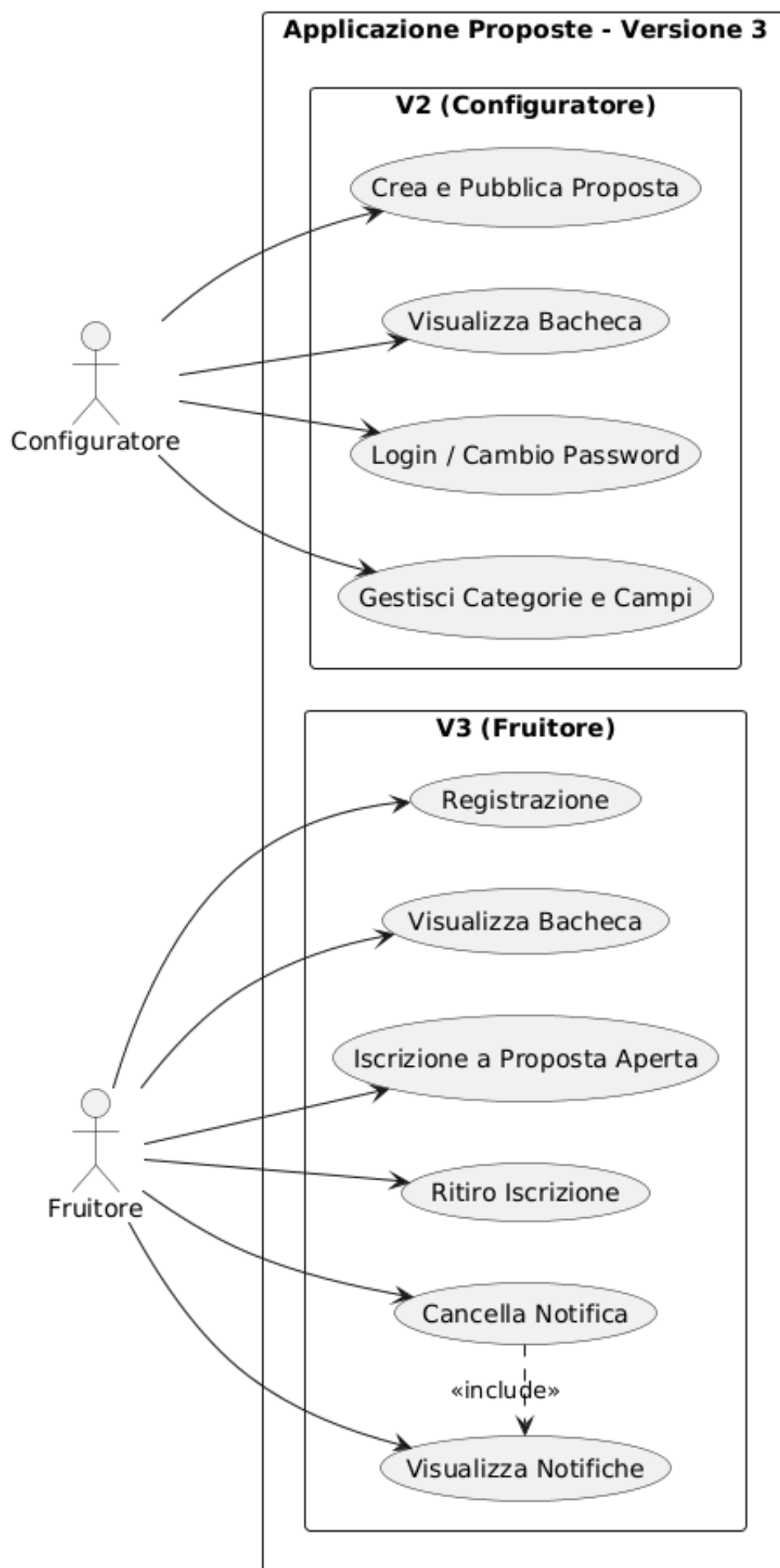
4.4.2 Nuove Classi (V3)

- **Fruitore extends Utente implements Observer**: aggiunge lista `notifiche`.

4.4.3 Metodi Principali Aggiunti

- `GestoreProposte.controllaScadenze(GestoreSessione)`: pianificatore che aggiorna stati e invia notifiche.
- `GestoreSessione.isFruitore()`: discrimina il tipo utente per il routing del menu.
- `MainCLI.menuFruitore()`: menu dedicato al Fruitore.
- `MainCLI.iscrivitiProposta()`: iscrizione con aggiunta automatica come observer.
- `MainCLI.ritiraIscrizione()`: disiscrizione con rimozione dall'observer list.

4.4.4 Diagramma delle Classi (Delta V3 - Pattern Observer)



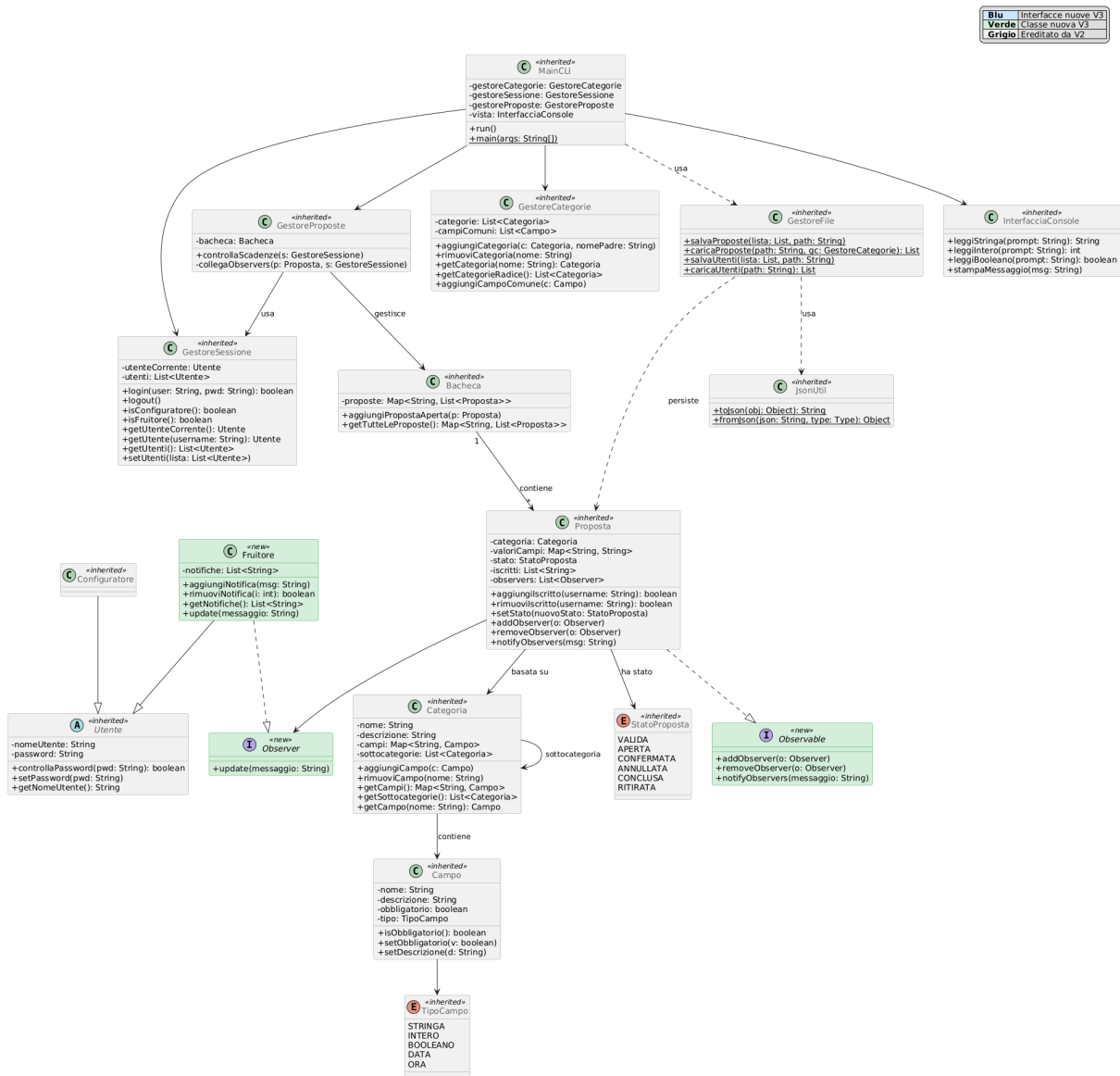


Figura 4.2: Diagramma delle Classi con Pattern Observer - Versione 3

Capitolo 5

Versione 4

5.1 Obiettivi e Requisiti

La quarta versione consolida la gestione del ciclo di vita delle proposte introducendo due nuove operazioni complementari: il **Ritiro** di un’iniziativa da parte del Configuratore per cause di forza maggiore, e la **Disiscrizione** formale da parte del Fruitore entro il termine utile.

5.2 Nuove Funzionalità (V4)

1. **Ritiro Proposta (Configuratore)**: Il configuratore può ritirare una proposta in stato APERTA o CONFERMATA, a condizione che non sia ancora iniziata (cioè la data dell’evento deve essere ancora futura). La proposta transisce allo stato RITIRATA e tutti i fruitori iscritti vengono automaticamente notificati via Observer.
2. **Disiscrizione Fruitore (formalizzata in V4)**: Il Fruitore può ritirare la propria iscrizione finché la proposta è in stato APERTA e il *Termine ultimo di iscrizione* non è ancora scaduto. Il sistema verifica esplicitamente questa condizione temporale.
3. **Persistenza dello Storico**: Le proposte ritirate (con la lista degli iscritti congelata) restano serializzate nel file JSON di bacheca, garantendo la tracciabilità storica delle iscrizioni.

5.3 Casi d’Uso (Aggiornamento V4)

- Ritiro Proposta Esistente (Configuratore)
- Disiscrizione entro scadenza (Fruitore)

5.4 Architettura e Design (Aggiunte rispetto a V3)

5.4.1 Nuovo Stato dell’Enum

- `StatoProposta.RITIRATA`: Stato terminale che indica il ritiro forzato dell’iniziativa.

5.4.2 Metodi Principali Aggiunti

- `GestoreProposte.ritiraProposta(Proposta, GestoreSessione)`: Verifica le condizioni temporali, richiama `collegaObservers()` per garantire che i Fruitori iscritti siano observer attivi, poi chiama `setStato(RITIRATA)` su `Proposta`, che propaga automaticamente la notifica via pattern Observer.
- `MainCLI.ritiraProposta()`: Mostra al Configuratore le proposte ritirabili (Aperte o Confermate), richiede conferma e delega a `GestoreProposte`.
- `MainCLI.ritiraIscrizione()` (aggiornata): Aggiunge controllo esplicito sul *Termine ultimo di iscrizione* prima di permettere la disiscrizione.

5.4.3 Diagramma delle Classi

Le modifiche in V4 riguardano i metodi esistenti e l'estensione dell'enum `StatoProposta`.

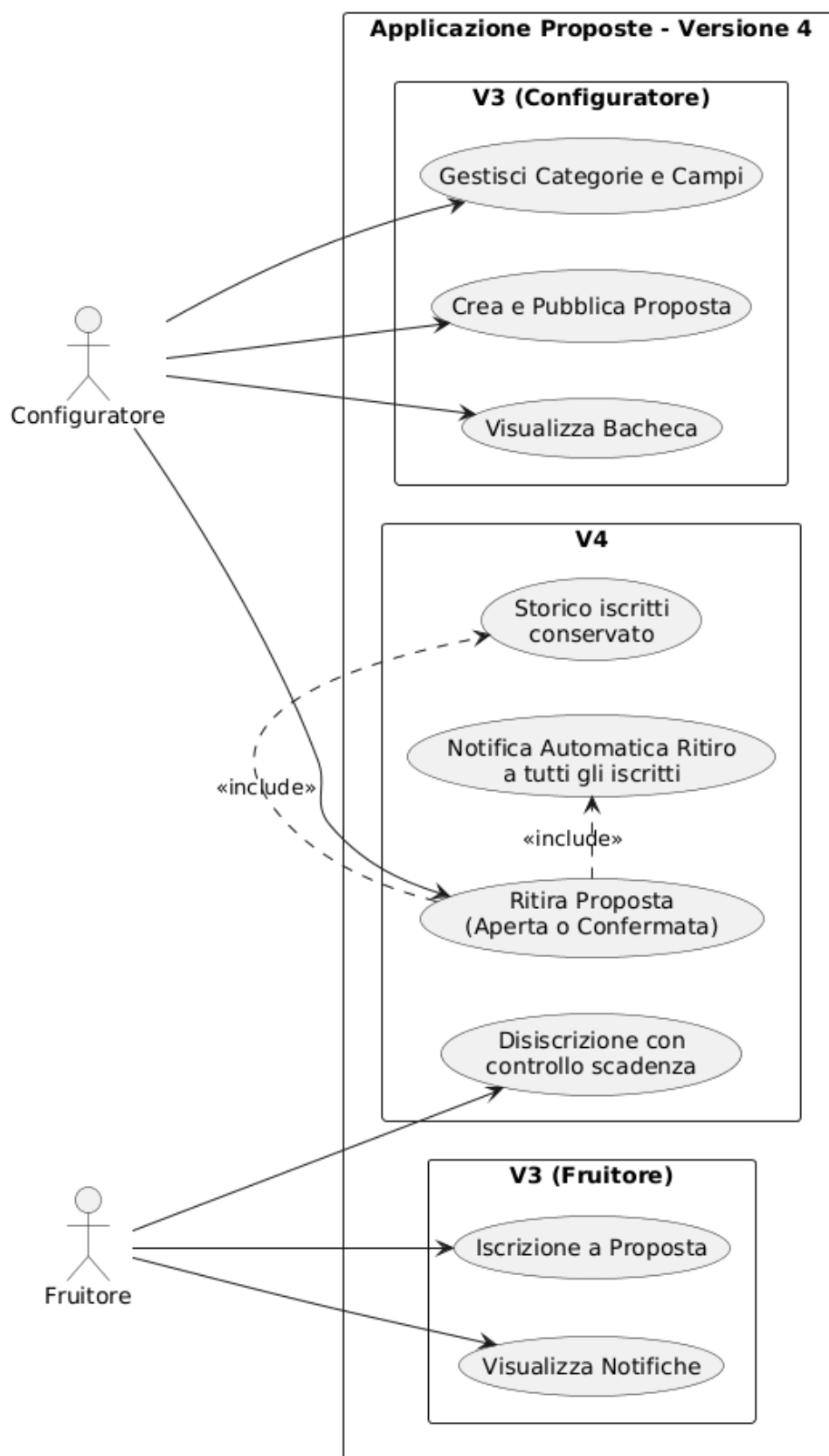


Figura 5.1: Diagramma dei Casi d'Uso - Versione 4

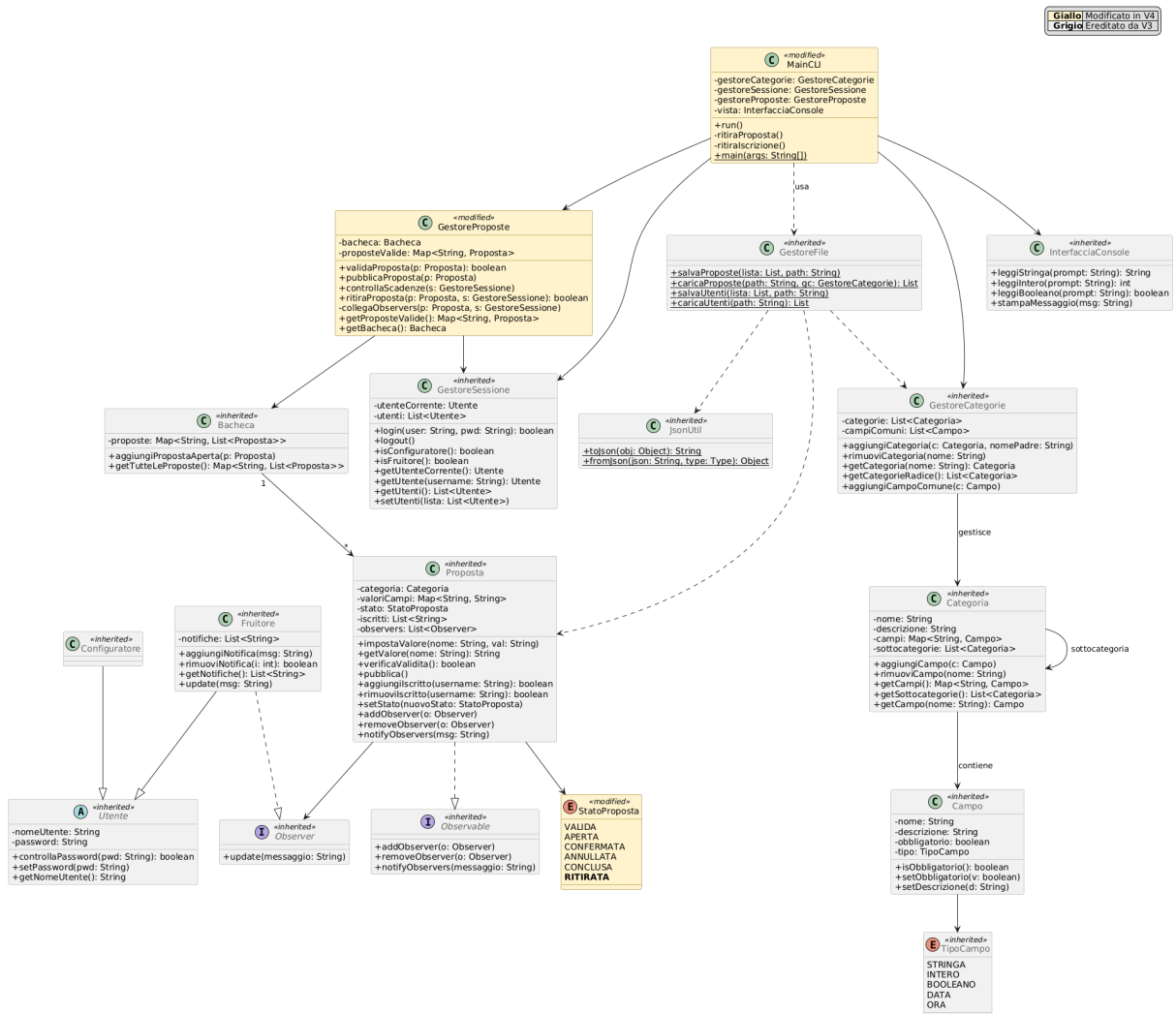


Figura 5.2: Diagramma delle Classi - Versione 4

Capitolo 6

Versione 5

6.1 Obiettivi e Requisiti

La quinta e ultima versione affianca alla modalità interattiva una **modalità batch**: il Configuratore può fornire un file di testo strutturato all'avvio, contenente comandi per creare massivamente categorie, campi e proposte senza passare per il menu.

6.2 Modalità Batch

6.2.1 Formato del File

Il file batch è un file di testo semplice (es. `batch.txt`) in cui ogni riga contiene un comando separato da punto e virgola (;). Le righe vuote e i commenti (che iniziano con #) vengono ignorati.

```
# Commento: crea una nuova categoria
```

```
CREA_CATEGORIA;NomeCategoria;Descrizione[;NomeParent]
```

```
# Aggiunge un campo alla categoria
```

```
AGGIUNGI_CAMPO;NomeCat;NomeCampo;Descrizione;OBBLIGATORIO|OPZIONALE;TIPO
```

```
# Crea una proposta (Data nel formato dd/MM/yyyy, Ora HH:mm)
```

```
CREA_PROPOSTA;Cat;Titolo;DataConclusiva;TermineIscrizione;Ora;Luogo;NumPart;Quota
```

```
# Pubblica una proposta valida già creata
```

```
PUBBLICA_PROPOSTA;Titolo
```

dove TIPO può essere: STRINGA, INTERO, BOOLEANO, DATA, ORA.

6.2.2 Esecuzione

Il file batch può essere utilizzato in due modi:

1. **Argomento da riga di comando** (all'avvio):

```
java -cp bin it.unibs.ing.view.MainCLI data/batch.txt
```

I comandi vengono elaborati prima che si avvii il loop interattivo.

2. **Voce 7 del menu Configuratore:** *Importa da File Batch*. Permette di specificare il percorso del file all'interno della sessione interattiva.

6.2.3 Output Batch

Per ogni comando, il sistema stampa [OK] o [ERRORE] con un messaggio descrittivo. Al termine, viene mostrato un riepilogo: *N OK, M ERRORI*.

6.3 Architettura (V5)

- `ImportatoreBatch` (NEW — `it.unibs.ing.storage`): legge il file riga per riga, effettua il parsing e delega le operazioni ai controller esistenti (`GestoreCategorie`, `GestoreProposte`). Tutti gli errori vengono gestiti localmente **senza interrompere** l'elaborazione delle righe successive.
- `GestoreProposte.getProposteValide()` (NEW): mappa temporanea in memoria per le proposte create ma non ancora pubblicate durante una sessione batch.
- `MainCLI.main()` (MODIFIED): accetta un argomento opzionale (`args[0]`) con il path del file batch, processato prima del loop interattivo.

6.4 Diagramma Globale dei Casi d'Uso

Qui sono illustrati tutti i casi d'uso cumulati delle versioni 1-4 più l'aggiunta dell'importazione batch (V5).

6.5 Diagramma Globale delle Classi

Il diagramma finale delle classi comprende l'intera struttura del sistema, inclusa la nuova classe per l'importazione batch.

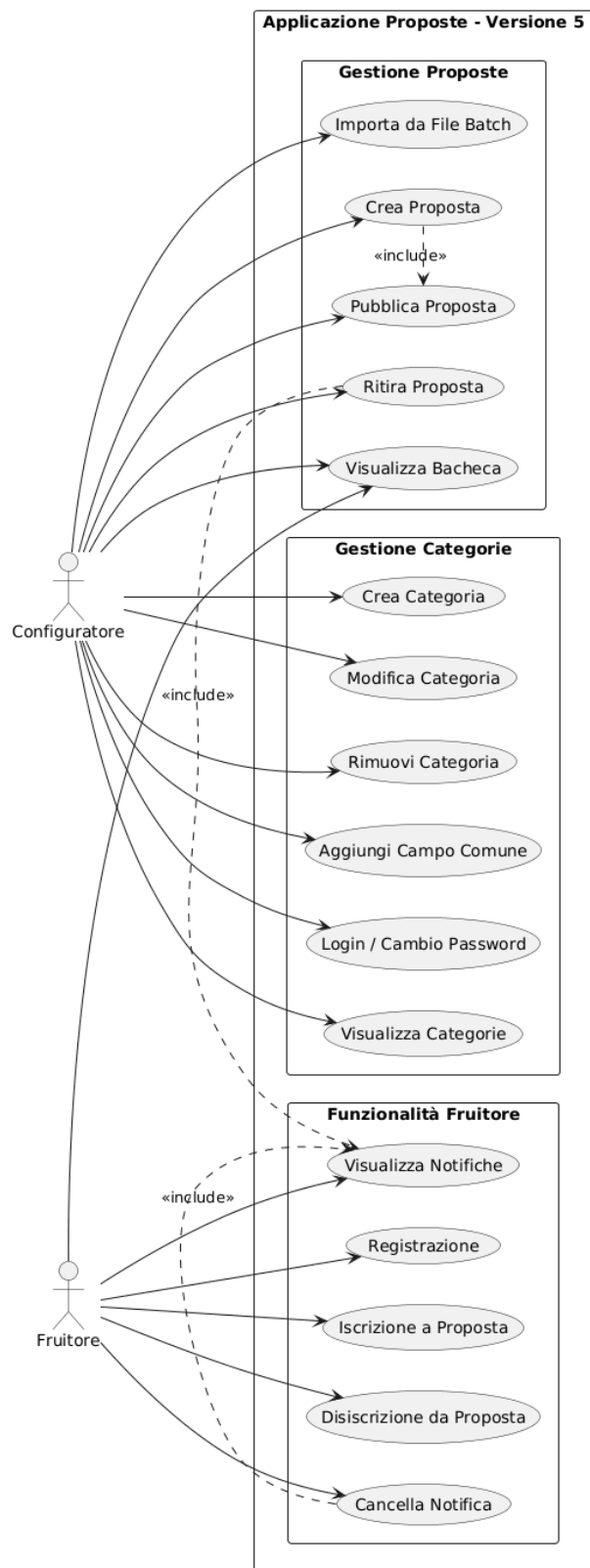


Figura 6.1: Diagramma Globale dei Casi d'Uso - Versione 5

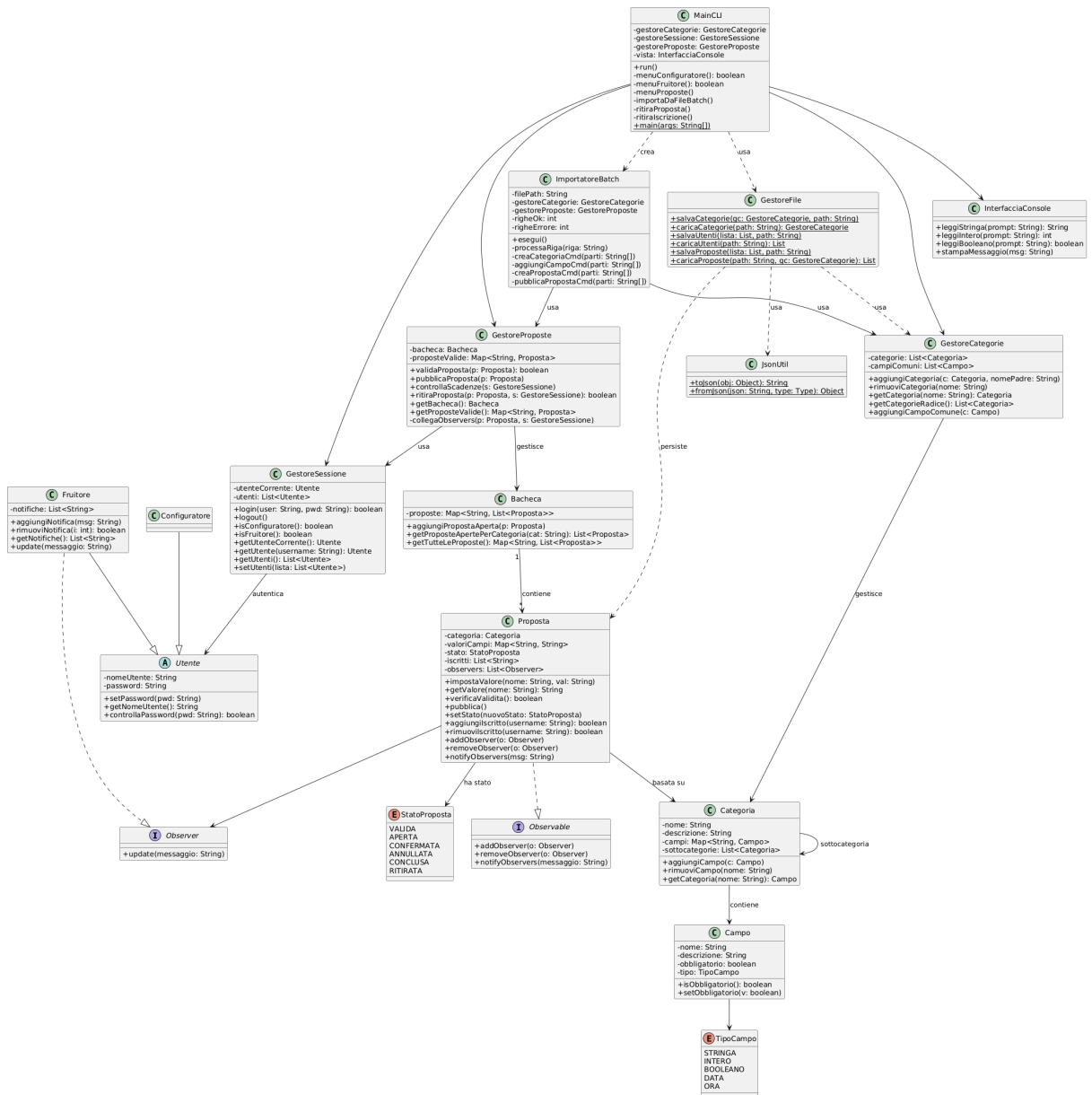


Figura 6.2: Diagramma delle Classi Finale - Versione 5

Capitolo 7

Manuale Utente

Questo capitolo raccoglie la manualistica necessaria all'utilizzo dell'ultima versione del software "Gestione Iniziative e Bacheca Eventi".

7.1 Installazione e Avvio

L'applicativo è progettato in puro linguaggio Java come interfaccia a linea di comando (CLI). Non richiede installazioni di server o database esterni, poichè poggia su un file system locale minimale tramite formati JSON.

7.1.1 Prerequisiti di Sistema

Per poter avviare correttamente il programma, è richiesto disporre di:

- **Java Development Kit (JDK)**: versione 17 o superiore.
- I file sorgenti del progetto posizionati in una directory locale sul proprio computer.

7.1.2 Struttura di Salvataggio Dati

Il software salva tutte le informazioni (categorie, proposte, e profili utenti) in una cartella denominata `data/` rispetto alla directory di esecuzione del progetto. **Nota:** Qualora questa cartella non sia presente (ad esempio, al primissimo avvio assoluto senza pacchetti di configurazione pre-esistenti), il codice la creerà automaticamente e popolerà i sistemi con asset di base al primo salvataggio.

7.1.3 Avvio tramite Ambiente di Sviluppo (IDE)

Il metodo più agevole ed immediato per i test è tramite IDE (Visual Studio Code, Eclipse, IntelliJ, ecc.):

1. Importare o aprire la cartella dell'ultima versione (`ProgettoIngSwP5`).
2. Posizionarsi sul pacchetto principale `it.unibs.ing.view`.
3. Avviare / Lanciare la classe `MainCLI.java`. L'applicazione interagirà tramite il terminale testuale integrato.

7.1.4 Avvio tramite Riga di Comando (Terminale)

Per chi desidera compilare e navigare il programma unicamente servendosi di prompt a riga di comando:

1. Posizionarsi all'interno della cartella di base del progetto (es. `cd c:/App/ProgettoIngSw/Progett`
2. Compilare i file sorgenti all'interno di una cartella destinazione, come `bin`:

```
javac -d bin src/it/unibs/ing/**/*.*.java
```

3. Invocare l'interprete Java sulla classe che contiene il Main:

```
java -cp bin it.unibs.ing.view.MainCLI
```

7.1.5 Avvio speciale con Importazione Batch (V5)

Sempre da riga di comando, è possibile lanciare il programma fornendo come parametro posizionale (argomento) il percorso di un file di test testuale per innescare la procedura di auto-compilazione da File Batch dei dati in fase di avvio pulito:

```
java -cp bin it.unibs.ing.view.MainCLI data/test_batch.txt
```

L'applicativo interpreterà i comandi testuali prima di aprire il Login.

7.2 Operatività e Manuale d'Uso

All'avvio, l'applicativo richiederà l'autenticazione tramite un prompt testuale. Esistono due ruoli principali: **Configuratore** (Amministratore) e **Fruitore** (Utente base). Di seguito viene spiegato come agire e quali funzionalità sono presenti nei rispettivi pannelli.

7.2.1 Primo Accesso al Sistema (Login e Registrazione)

- **Accesso come Configuratore:** Se si avvia il sistema per la prima volta (nessun salvataggio precedente), è necessario effettuare il login con *Nome Utente*: **admin** e *Password*: **admin**. Il sistema riconoscerà il primo accesso di default e forzerà l'utente a cambiare la password inserendone una nuova (diversa da "admin").
- **Registrazione come Fruitore:** Al prompt del Login, è sufficiente digitare un qualsiasi nome utente desiderato. Poiché il sistema non lo troverà tra gli utenti registrati, domanderà: *"Vuoi registrarti come nuovo Fruitore con questo nome utente?"*. Scrivendo **si** (o **true** in base all'input), chiederà l'immissione di una password e l'account Fruitore sarà istantaneamente creato e validato.

7.2.2 Menu del Configuratore (Amministratore)

Una volta autenticato come Configuratore, viene presentato un Menu principale. Digittando l'Option number desiderato e premendo Invio, si accede alle funzioni di gestione:

1. **1. Visualizza Categorie:** Stampa a schermo l'elenco e l'albero gerarchico completo delle Categorie esistenti, esplodendo la vista anche delle relative *Sottocategorie* e indicando quanti campi specifici possiedono.
2. **2. Crea Nuova Categoria:** Avvia il wizard di creazione. Il sistema richiederà in sequenza: *Nome*, *Descrizione*, chiedendo in seguito se debba essere legata come Sottocategoria ad una categoria preesistente (indicando il nome del Padre). Infine, la procedura chiederà ciclicamente all'utente se desidera inserire nuovi **Campi Specifici**. Per ogni campo domanderà il Nome, la Descrizione, l'Obbligatorietà e il Tipo (da 0 a 5 per scegliere Stringa, Intero, Booleano, Data, Ora, Double).
3. **3. Aggiungi Campo Comune:** Permette di dichiarare un campo che verrà forzatamente innestato contemporaneamente all'interno di *tutte* le categorie strutturali. Richiede Nome, Tipo e Obbligatorietà.
4. **4. Rimuovi Categoria:** Inserendo il nome esatto della Categoria, il sistema provvederà ad eliminarla dall'albero (se possibile in base ai vincoli in essere).
5. **5. Modifica Categoria:** Offre la possibilità di selezionare una Categoria ed entrare in modifica di uno dei suoi Campi Specifici: è possibile rettificarne la Descrizione, l'Obbligatorietà, o rimuovere il campo stesso dalla struttura.
6. **6. Gestisci Proposte (Sottomenu):**
 - *6.1 Crea Nuova Proposta:* Chiede di selezionare in quale Categoria creare l'iniziativa e farà compilare i campi uno per uno. Al termine validerà severamente le Date (es. la data limite d'iscrizione deve precedere di almeno due giorni l'inizio) e lascerà scegliere se pubblicarla subito.
 - *6.2 Pubblica Proposta:* Permette di inserire in Bacheca le proposte tenute "Lavorazione" o ritirate in precedenza.
 - *6.3 Visualizza Bacheca:* Mostra la vetrina completa delle Proposte Aperte visibili al pubblico.
 - *6.4 Ritira Proposta:* In caso di emergenza o cause di forza maggiore, l'immissione di questo comando passa lo stato della proposta in **Ritirata**. A livello logico invaliderà le iscrizioni, avviserà i Partecipanti, pur conservando uno storico dei dati.
7. **7. Importa da File Batch:** Richiede il path ad un file `.txt` a riga singola (es. `data/test_batch.txt`) per simulare una rapida configurazione massiva di categorie e proposte.
8. **8. Logout / 9. Salva & Esci:** La funzione di Logout rimanda l'utente al Login schermandolo dai dati. "Salva & Esci" compie la medesima azione del logout finale, ma si assicura tassativamente che l'intera mappa persistente venga serializzata nei file `.json`.

7.2.3 Menu del Fruitore (Utente Base)

Effettuando il Login nel ruolo del fruitore, l'utente avrà accesso interattivo alle funzioni della Bacheca Eventi.

1. **1. Visualizza Bacheca:** Enumera sul prompt tutte le iniziative esposte in stato **APERTA**. Ne mostra Titolo, Luogo, Informazioni e la capienza (posti occupati/posti totali previsti).
2. **2. Iscriviti a una Proposta:** Permette all'utente di selezionare una Categoria e poi di scegliere con numero puntuale la proposta alla quale prender parte. Se la capienza del *Numero Partecipanti* non è stata raggiunta, viene effettuata l'associazione utente-evento in database.
3. **3. Ritira l'Iscrizione:** Mostra al Fruitore l'elenco unicamente dei propri impegni futuri. Permette di annullare la propria presenza per un determinato evento a patto di ritirare l'iscrizione anticipatamente rispetto alla *Data ultima di Iscrizione*.
4. **4. Area Personale (Notifiche):** La casella Inbox dell'utente. All'avvio del programma o al cambio di giorno, il **GestoreProposte** scorre tutte le scadenze: se una Proposta raggiunge il quorum di partecipanti, passa di stato ed avvisa via design pattern Observer inviando una notifica scritta qui in Area Personale a tutti gli acquirenti.
5. **5. Logout / 6. Salva & Esci:** Rientro all'area di autenticazione o spegnimento in scrittura sul filesystem Json.